Beginner's Toolkit with GenAI

Getting Started with React for Full-Stack ML Web Applications

Rose Kyalo | Moringa School AI Capstone | 26th March 2026

---

1. Title & Objective

Technology chosen: React (JavaScript library) + FastAPI (Python backend)

Why this technology?

As a data scientist with experience building end-to-end machine learning systems, I realized that a model alone is not enough. In a project predicting financial distress for startups, the model lived in a Jupyter notebook invisible to the startups who needed it most. React gives me the frontend skills to turn ML outputs into usable, interactive products.

End goal:

Build a full-stack web application called the FinDistress Predictor, where users input financial indicators and receive a distress risk score. The frontend is built in React, the backend in FastAPI, and the architecture is ready to plug in a real LightGBM model.

---

2. Quick Summary of the Technology

React

React is a JavaScript library for building user interfaces, created by Facebook. It uses a component-based model where each piece of the UI is an independent, reusable function. When data changes, React automatically updates only the parts of the page that need to change.

Where it is used:

- Web dashboards, data visualization tools, interactive ML demos
- Production apps: Facebook, Airbnb, Netflix, Notion
- ML product interfaces: model prediction forms, result displays, SHAP charts

Real-world example: A startup founder enters financial data into a form. React sends it to the backend, receives a prediction, and re-renders the result card without reloading the page.

FastAPI

FastAPI is a modern Python web framework for building APIs. It is fast, supports async, auto-generates documentation, and integrates directly with scikit-learn, LightGBM, and joblib making it ideal for serving ML models.

---

3. System Requirements

Operating System: Windows / macOS / Linux

Required tools:

- Node.js v18+ (includes npm)
- Python 3.8+
- pip
- VS Code (recommended editor)
- A terminal (PowerShell, bash, or zsh)

---

4. Installation & Setup

Frontend (React + Vite)

1. Create the project:

```
npm create vite@latest finstress-app -- --template react
```

2. Install dependencies:

```
cd finstress-app
npm install react react-dom
npm install
```

3. Start the dev server:

```
npm run dev
```

Opens at http://localhost:5173

Backend (FastAPI)

4. Create backend folder:

```
mkdir backend && cd backend
```

5. Install packages:

```
pip install fastapi uvicorn
```

6. Start the server:

```
uvicorn main:app --reload
```

Runs at http://localhost:8000

---

5. Minimal Working Example

The FinDistress Predictor takes four financial inputs from a user i.e company name, annual revenue, debt ratio, and cash flow. It then sends them to a FastAPI backend, and displays a distress probability score with a risk label and contributing factors.

Project structure

```
finstress-app/
  src/
    components/
      PredictionForm.jsx   # Form with 4 inputs
      ScoreCard.jsx        # Displays score + factors
    App.jsx                # Root component
    main.jsx               # React entry point
    style.css              # All styles
backend/
  main.py                  # FastAPI app + prediction logic
```

Key code: App.jsx

App.jsx holds the result state and passes it between the form and the score card:

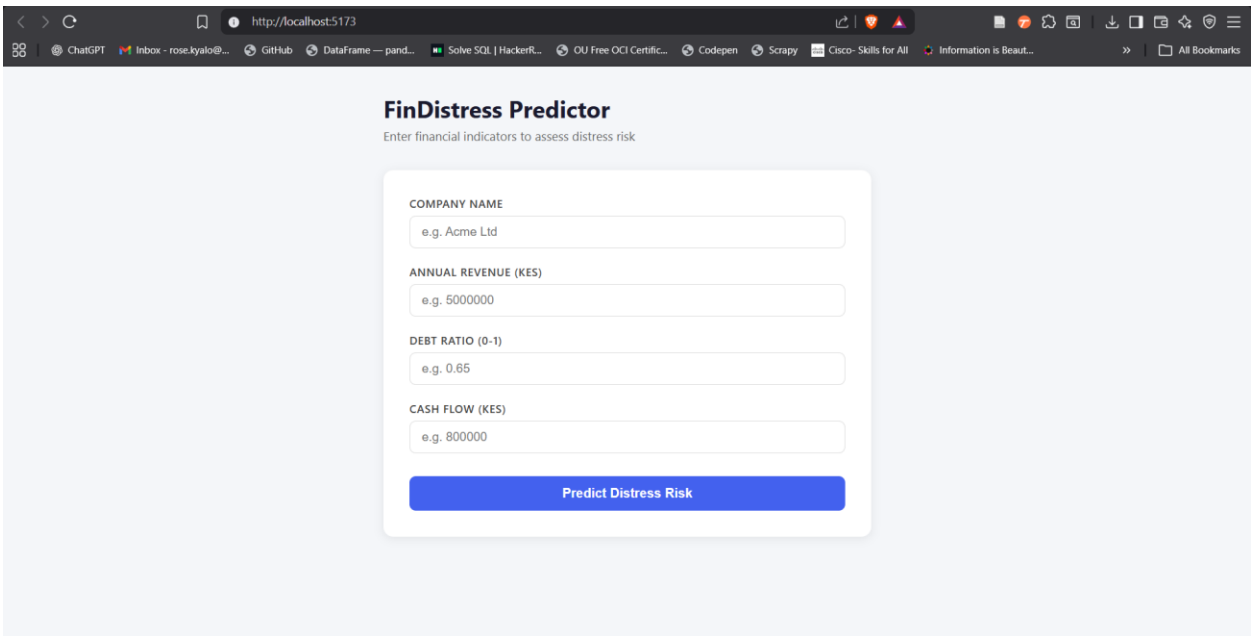
```
function App() {
  const [result, setResult] = useState(null)
  return (
    <div className='app'>
      <PredictionForm onResult={setResult} />
      {result && <ScoreCard result={result} />}
    </div>
  )
}
```

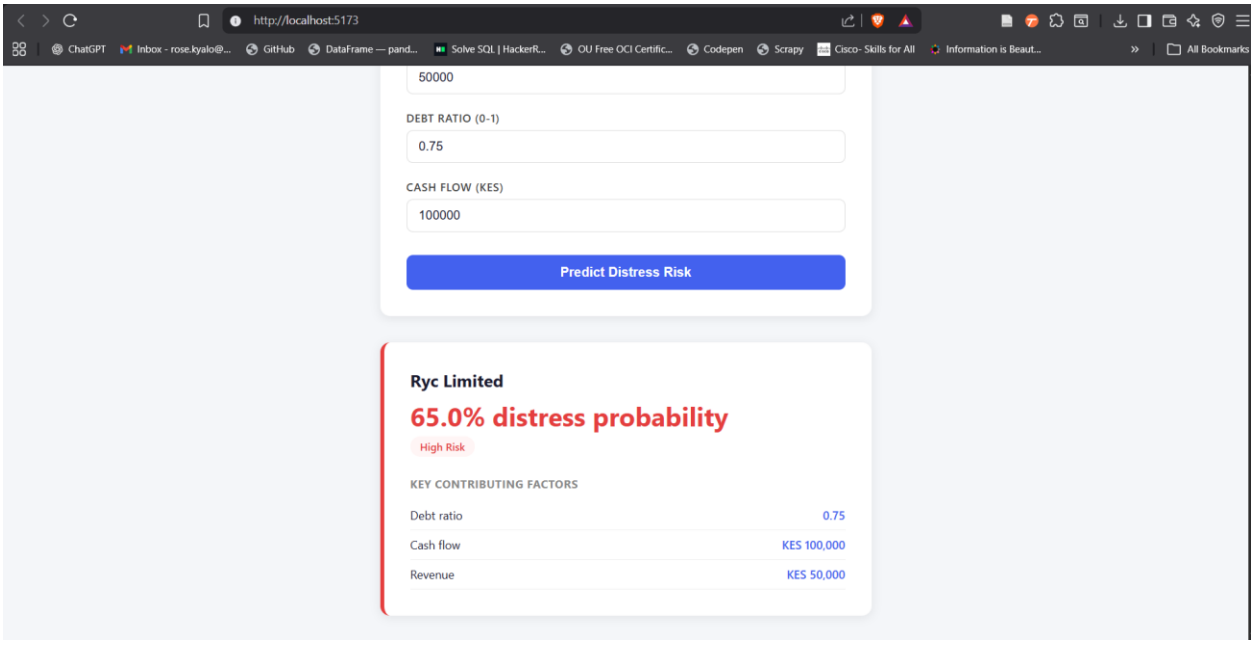
Key code: FastAPI backend (main.py)

The backend receives JSON, runs a rule-based scoring function, and returns a prediction:

```
@app.post('/predict')
def predict(data: CompanyData):
    score = mock_predict(data)
    return { 'company_name': data.company_name,
             'score': score, 'factors': [...] }
```

Expected output: A score card showing e.g. '40.0% distress probability' with a green Low Risk or red High-Risk label, and a breakdown of the three input factors.





---

6. AI Prompt Journal

Prompt 1: Learning React concepts

Prompt used:

"I would like to learn React and build a simple frontend and backend web application. My interest comes from my experience as a data scientist. Below is my learning plan. I would like your help guiding me through learning React in a practical, project-based way."

AI response summary:

The AI provided plain-language explanations of React's core concepts — components, JSX, state, and props — with data science analogies. For example: React components are like scikit-learn Pipeline steps — modular, composable, and swappable. State is like a reactive cell in a spreadsheet; when it changes, everything depending on it updates automatically.

Evaluation:

Highly effective. Framing React through Python and data science concepts I already knew made the mental model click faster than learning it cold. The analogy between JSX curly braces and Python f-strings was particularly useful.

Prompt 2: Scaffolding the project

Prompt used:

"Help me understand what is expected of this capstone project. I want to build the working codebase first. Not started yet. Keeping it simple to understand."

AI response summary:

The AI provided the full project explanation step by step with practical analogies.

Evaluation:

Very helpful for getting unstuck quickly.

Prompt 3: Debugging rendering issues

Prompt used:

"The application is still not rendering "Still showing Vite's default demo", here is the code and the error I am getting. Help me understand it "

AI response summary:

Through a series of diagnostic steps (checking file contents with cat, reading the directory with ls), the AI identified three separate issues: wrong CSS import path, Vite creating a vanilla JS project instead of React, and the entry div having id='app' instead of id='root'.

Evaluation:

The structured debugging approach i.e check the files before assuming what is wrong, was the most valuable part. It taught a debugging methodology, not just a fix.

---

7. Common Issues & Fixes

Error	Cause	Fix
Vite default page still showing	App.jsx not yet replacing main.jsx entry	Replace src/main.js with React bootstrap code; rename to main.jsx

Failed to parse JSX — invalid JS syntax	File named main.js but contains JSX	Rename to main.jsx; update index.html script src
Blank page, no errors in terminal	index.html has id='app', React expects id='root'	Change to in index.html
Cannot find module './App.css'	Vite created style.css, not App.css	Change import './App.css' to import './style.css' in App.jsx
CORS error when calling FastAPI	Browser blocks cross-origin requests by default	Add CORSMiddleware to FastAPI allowing localhost:5173
Encoding error: Debt Ratio (0â€"1)	Em dash pasted with wrong encoding	Replace with plain hyphen: Debt Ratio (0-1)

---

8. References

Official Documentation

- React — https://react.dev
- Vite — https://vite.dev
- FastAPI — https://fastapi.tiangolo.com
- MDN Web Docs (JavaScript) — https://developer.mozilla.org

Further Learning

- React tutorial (official) — https://react.dev/learn
- FastAPI tutorial — https://fastapi.tiangolo.com/tutorial
- Full Stack Open (free course) — https://fullstackopen.com
- SHAP documentation — https://shap.readthedocs.io